

## PROYECTO PRX

# Informe técnico de Notas y Bitácora

## Código implementado calidad SOLID patrones y defensa

Este documento explica el mantenimiento realizado en los módulos de Notas y Bitácora. Detalla las funciones añadidas, el recorrido de los datos desde la interfaz hasta MySQL, las decisiones de diseño y la forma de defender el trabajo según puntualidad, calidad, principios SOLID y patrones de diseño.

| Aspecto      | Contenido                                                              |
|--------------|------------------------------------------------------------------------|
| Notas        | Enlaces URL, tareas, imágenes y editor de imágenes                     |
| Bitácora     | Enlaces URL, tareas, vista separada de edición y guardado del checkbox |
| Frontend     | Angular 21, Signals, formularios reactivos, PrimiIcons y Fabric.js     |
| Backend      | NestJS, CQRS, DTO, validación, permisos y repositorios                 |
| Persistencia | Prisma, MySQL, campos JSON, archivos privados y borrado lógico         |
| Verificación | Build Angular y build NestJS ejecutados el 23 de septiembre de 2026    |

## Resultado principal

Las notas dejaron de ser únicamente título y contenido: ahora relacionan recursos web, manejan tareas, adjuntan imágenes y permiten modificarlas antes de guardarlas. Las bitácoras también administran enlaces y tareas; su consulta está separada de la edición y el usuario puede completar una tarea directamente desde la vista.

# 1 Alcance realizado

| Módulo   | Función     | Comportamiento entregado                                                         |
|----------|-------------|----------------------------------------------------------------------------------|
| Notas    | Enlaces URL | Agregar, normalizar, validar, listar, abrir y eliminar enlaces.                  |
| Notas    | Tareas      | Agregar, eliminar, mostrar y marcar o desmarcar con persistencia.                |
| Notas    | Imágenes    | Adjuntar JPEG, PNG y WebP, previsualizar, reemplazar y eliminar.                 |
| Notas    | Editor      | Seleccionar, dibujar, marcar, recortar, rotar, escribir, usar formas y deshacer. |
| Bitácora | Enlaces URL | Crear y editar listas de URL HTTP o HTTPS.                                       |
| Bitácora | Tareas      | Crear, eliminar y cambiar el estado desde la vista o edición.                    |
| Bitácora | Detalle     | Abrir una página de consulta y entrar a Editar solo cuando se necesita.          |

## 2 Tecnologías que usamos

| Tecnología      | Dónde        | Por qué se usó                                                               |
|-----------------|--------------|------------------------------------------------------------------------------|
| Angular 21      | Frontend     | Componentes standalone, rutas, formularios y renderizado de las pantallas.   |
| Signals         | Frontend     | Estado reactivo de tareas, enlaces, archivos, carga, edición y guardado.     |
| Reactive Forms  | Frontend     | Validación de título y contenido antes de enviar datos.                      |
| Fabric.js       | Editor       | Canvas, dibujo, texto, figuras, selección, recorte e historial.              |
| FormData        | Notas API    | Enviar en una misma petición texto, JSON y archivos binarios.                |
| NestJS          | Backend      | Controladores, inyección de dependencias, validación y organización modular. |
| CQRS            | Backend      | Separar Commands que modifican datos de Queries que los consultan.           |
| class validator | DTO          | Rechazar tareas, enlaces y campos con formato o tamaño incorrectos.          |
| Prisma          | Persistencia | Acceso tipado a MySQL y mapeo de campos JSON.                                |
| Tigris Storage  | Archivos     | Guardar imágenes y adjuntos privados fuera de la base de datos.              |
| JWT y Roles     | Seguridad    | Identificar al usuario y comprobar permisos antes de leer o modificar.       |

## 3 Implementación de Notas

La implementación de Notas se distribuyó entre presentación, contratos de dominio, API, casos de uso y persistencia. Esta separación evita que la pantalla tenga que conocer Prisma, MySQL o el almacenamiento.

### 3.1 Modelos de tareas y enlaces

```
export interface NoteTaskModel {
  id: string;
  title: string;
  completed: boolean;
}

export interface NoteLinkModel {
  id: string;
  url: string;
}
```

**Qué hace** Define la forma exacta de cada tarea y enlace en Angular.

**Por qué se añadió** Antes la nota no tenía contratos tipados para estas listas. El id permite localizar cada elemento, completed conserva el estado y url guarda la dirección normalizada.

### 3.2 Enlaces URL

- El usuario puede escribir una URL con o sin protocolo.
- normalizeUrl agrega https cuando falta el protocolo.
- isValidUrl usa la clase URL del navegador y restringe el resultado a HTTP o HTTPS.
- Cada enlace obtiene un identificador con crypto.randomUUID.
- La vista abre enlaces con target blank y rel noopener noreferrer para reducir riesgos.

```
private normalizeUrl(value: string): string {
  const trimmed = value.trim();
  if (!trimmed) return '';
  return /^https?:\/\//i.test(trimmed) ? trimmed : `https://${trimmed}`;
}

private isValidUrl(value: string): boolean {
  try {
    const url = new URL(value);
    return url.protocol === 'http:' || url.protocol === 'https:';
  } catch {
    return false;
  }
}
```

### 3.3 Tareas y guardado desde la vista

```
protected toggleTask(task: NoteTaskModel): void {
  if (!this.canEdit() || this.saving()) return;
  this.tasks.update((tasks) => tasks.map((item) =>
    item.id === task.id ? { ...item, completed: !item.completed } : item
  ));
  if (!this.editing()) this.persist(false);
}
```

La lista se actualiza de forma inmutable. Cuando el usuario está en modo Ver, persist false envía el cambio sin abrir ni cerrar el editor. El bloqueo saving evita peticiones duplicadas mientras una actualización está en curso.

### 3.4 Imágenes y archivos

- Se separan imágenes de otros adjuntos mediante el tipo MIME.
- Se aceptan JPEG, PNG y WebP y se comprueba el tamaño máximo.
- La nota mantiene un máximo total de cinco archivos entre existentes y nuevos.
- URL.createObjectURL crea una previsualización local y revokeObjectURL libera memoria.
- Una imagen ya guardada se descarga temporalmente, se convierte en File y puede volver al editor.
- Al guardar una edición se reemplaza el archivo original por una nueva imagen PNG.

### 3.5 Editor de imágenes

| Herramienta | Código o API                  | Resultado                                   |
|-------------|-------------------------------|---------------------------------------------|
| Seleccionar | Canvas selection              | Mover, redimensionar y seleccionar objetos. |
| Pincel      | PencilBrush                   | Dibujar con color y grosor configurables.   |
| Marcador    | PencilBrush con transparencia | Resaltar zonas sin ocultar la imagen.       |
| Recortar    | Rect y Canvas toDataURL       | Exportar únicamente el área seleccionada.   |
| Texto       | IText                         | Insertar y editar texto sobre la imagen.    |
| Formas      | Rect Circle Triangle          | Añadir rectángulo, círculo y flecha.        |
| Rotar       | Canvas 2D rotate              | Girar el resultado 90 grados.               |
| Historial   | toJSON y loadFromJSON         | Deshacer y rehacer hasta 30 estados.        |
| Guardar     | canvas toBlob                 | Emitir un archivo PNG con nombre editada.   |

```
const blob = await this.canvas.toBlob({ format: 'png', multiplier: 1 });
this.saved.emit(new File([blob], `${baseName}-editada.png`, {
  type: 'image/png',
  lastModified: Date.now(),
}));
```

### 3.6 Transporte al backend

```
formData.append('tasks', JSON.stringify(data.tasks));
formData.append('links', JSON.stringify(data.links));
formData.append('retainedFileIds', data.retainedFileIds.join(','));
files.forEach((file) => formData.append('files', file, file.name));
```

Se usó FormData porque una actualización de nota puede combinar campos de texto, arreglos JSON y archivos. El backend vuelve a analizar las listas; no confía únicamente en la validación del navegador.

### 3.7 Validación permisos y persistencia

- UpdateNoteHandler verifica que la nota y el repositorio existan.
- En repositorios íntimos solo el propietario puede modificar; en otros se acepta propietario o miembro.
- parseTasks limita a 100 tareas, exige id y título y recorta cada título a 200 caracteres.
- parseLinks limita a 50 enlaces, analiza cada URL y limita su longitud a 2048 caracteres.
- Los archivos eliminados usan soft delete y también se retiran del almacenamiento privado.
- Prisma guarda tasks y links como JSON y devuelve la nota completa mediante un Mapper.

## 4 Implementación de Bitácora

Bitácora reutiliza la misma estructura conceptual de tareas y enlaces, pero transporta JSON normal en lugar de FormData porque no incluye archivos. Se añadió una página de detalle para separar claramente consulta y edición.

### 4.1 Crear una bitácora con tareas y enlaces

```
export interface CreateBinnacleRequest {
  name: string;
  content: string;
  tasks: { id: string; title: string; completed: boolean }[];
  links: { id: string; url: string }[];
}
```

El formulario de creación envía el título, el contenido y ambas listas en una sola petición. El Handler recorta espacios y construye BinnacleEntity antes de llamar al repositorio.

### 4.2 Vista separada de edición

- La ruta de detalle obtiene el id y carga la bitácora con GET por id.
- En modo Ver se muestran contenido, fechas, enlaces y tareas; los campos de edición permanecen ocultos.
- El botón Editar cambia la señal editing y muestra controles para título, contenido, enlaces y tareas.
- Cancelar restablece los valores originales de la bitácora y descarta los cambios locales.
- Guardar valida el formulario, agrega entradas pendientes y ejecuta PUT por id.

### 4.3 Marcar tareas desde Ver

```
protected toggleTask(id: string): void {
  if (this.saving()) return;
  this.tasks.update((tasks) =>
    tasks.map((task) => task.id === id
      ? { ...task, completed: !task.completed }
      : task),
  );
  if (!this.editing()) this.persist(false);
}
```

Este comportamiento cumple la solicitud de poder completar tareas sin entrar a Editar. La interfaz cambia el checkbox y llama a persist false. El backend comprueba que el userId del JWT sea el propietario antes de actualizar.

## 4.4 Validaciones del backend

| Dato            | Regla                      | Objetivo                                     |
|-----------------|----------------------------|----------------------------------------------|
| Título          | 2 a 15 caracteres          | Conservar el contrato existente de Bitácora. |
| Contenido       | 1 a 2000 caracteres        | Evitar datos vacíos o excesivos.             |
| Tareas          | Arreglo de máximo 50       | Controlar volumen y validar cada elemento.   |
| Título de tarea | No vacío y máximo 250      | Mantener tareas legibles.                    |
| Enlaces         | Arreglo de máximo 50       | Controlar volumen de referencias.            |
| URL             | HTTP o HTTPS con protocolo | Rechazar esquemas y formatos no esperados.   |
| Permiso         | Rol estándar y propietario | Impedir editar bitácoras ajenas.             |

## 4.5 Código del caso de uso

```

if (binnacle.userId !== command.userId) {
  throw new ForbiddenException(BINNACLE_MESSAGES.FORBIDDEN);
}

const updated = await this.binnacleRepository.update(command.id, {
  name: command.dto.name.trim(),
  content: command.dto.content.trim(),
  tasks: command.dto.tasks.map((task) => ({ ...task, title: task.title.trim() })),
  links: command.dto.links.map((link) => ({ ...link, url: link.url.trim() })),
  updatedBy: command.userId,
});

```

# 5 Base de datos y recorrido de los datos

## 5.1 Campos agregados

```

model Note {
  tasks Json?
  links Json?
}

model Binnacle {
  tasks Json?
  links Json?
}

```

Sí existen campos para guardar tareas y enlaces. Se usaron columnas JSON opcionales porque cada lista pertenece a una sola nota o bitácora y su estructura es pequeña. Prisma convierte los arreglos a `InputJsonValue` al crear o actualizar.

## 5.2 Flujo completo

1. El usuario agrega una tarea, URL o imagen en Angular.
2. El componente valida y actualiza Signals sin mutar el arreglo anterior.
3. La Facade coordina la llamada y la API construye JSON o FormData.
4. El Controller extrae el usuario del JWT y envía un Command al CommandBus.
5. El DTO valida formato y límites; el Handler valida permisos y reglas del caso de uso.
6. El Repository usa Prisma para guardar tasks y links como JSON en MySQL.
7. El Mapper transforma el registro de Prisma en entidad y luego en DTO de respuesta.
8. El Store y el componente actualizan la interfaz con el resultado confirmado por el servidor.

## 6 Código incorporado por capa

| Capa                  | Archivos principales               | Responsabilidad añadida                           |
|-----------------------|------------------------------------|---------------------------------------------------|
| Presentación Notas    | create note page note detail page  | Controles URL, tareas, imágenes, vista y edición. |
| Editor Notas          | note image editor component        | Canvas Fabric.js y herramientas gráficas.         |
| Dominio Notas         | note model create y update request | Contratos de Task, Link y archivos retenidos.     |
| API Notas             | note api                           | FormData con tasks, links y archivos.             |
| Backend Notas         | DTO Handler Entity                 | Parseo, permisos, reglas y normalización.         |
| Persistencia Notas    | Mapper PrismaNoteRepository        | Lectura y escritura de JSON.                      |
| Presentación Bitácora | binnacle page detail page          | Crear, ver, editar y marcar tareas.               |
| Dominio Bitácora      | model requests contracts           | Contratos de Task y Link.                         |
| Backend Bitácora      | DTO Controllers Handlers           | Rutas por id, validación y propiedad.             |
| Persistencia Bitácora | Mapper PrismaBinnacleRepository    | Guardar y recuperar JSON.                         |
| Esquema               | prisma schema                      | Campos tasks y links en Note y Binnacle.          |

## 7 Calidad del trabajo

### 7.1 Evidencia comprobada

| Comprobación  | Resultado | Lectura para la defensa                              |
|---------------|-----------|------------------------------------------------------|
| Build Angular | Correcto  | La aplicación frontend genera el bundle sin errores. |

| Comprobación   | Resultado             | Lectura para la defensa                                      |
|----------------|-----------------------|--------------------------------------------------------------|
| Build NestJS   | Correcto              | Prisma Client se genera y TypeScript compila sin errores.    |
| Validación URL | Frontend y backend    | No se depende de una sola capa para validar datos.           |
| Permisos       | Propiedad o membresía | Los cambios no exponen datos de otros usuarios.              |
| Persistencia   | Campos JSON           | Los enlaces y estados de tareas permanecen tras recargar.    |
| Archivos       | Límite y tipos        | Máximo cinco archivos y formatos de imagen definidos.        |
| Guardado       | Estados saving        | Se evita repetir acciones mientras una petición está activa. |

## 7.2 Detalles que demuestran calidad

- Los inputs tienen límites coherentes con los DTO y el esquema de base de datos.
- Los errores HTTP se transforman en notificaciones comprensibles para el usuario.
- Los enlaces externos usan noopener noreferer.
- El editor libera URL temporales y destruye Canvas y ResizeObserver al cerrar.
- Los Mappers devuelven arreglos vacíos cuando un campo JSON es nulo o antiguo.
- El borrado es lógico mediante status igual a cero para conservar auditoría.

## 7.3 Aspectos pendientes

- No se encontraron pruebas automatizadas específicas para las nuevas funciones; deben añadirse pruebas unitarias y de integración.
- El build Angular muestra una advertencia: create note page component SCSS supera el presupuesto de 5 KB por 818 bytes.
- Conviene probar archivos grandes, fallos de red y edición simultánea antes de producción.

Mencionar estas mejoras no invalida el Sprint. Demuestra que se conoce el estado real del producto y que existe un plan de calidad continuo.

## 8 Principios SOLID aplicados

| Principio                   | Dónde se observa                                | Por qué es útil                                                                       |
|-----------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------|
| S Responsabilidad única     | Component Facade API Handler Repository Mapper  | Cada clase se concentra en interfaz, coordinación, HTTP, negocio, datos o conversión. |
| O Abierto cerrado           | Contratos y componentes aislados                | Se amplían capacidades sin reescribir todas las capas consumidoras.                   |
| L Sustitución de Liskov     | PrismaNoteRepository implementa NoteRepository  | La aplicación usa el contrato sin depender del detalle de Prisma.                     |
| I Segregación de interfaces | NoteApiContract y BinnacleApiContract separados | Cada módulo depende solo de las operaciones que necesita.                             |

| Principio                   | Dónde se observa                       | Por qué es útil                                              |
|-----------------------------|----------------------------------------|--------------------------------------------------------------|
| D Inversión de dependencias | Handlers inyectan Repository abstracto | El caso de uso no crea ni conoce directamente PrismaService. |

## Cómo defender SOLID

No colocamos todo el código en el componente. La pantalla administra interacción; la Facade coordina; la API transporta; el DTO y el Handler validan; el Repository persiste; el Mapper traduce formatos. Si mañana cambiamos Prisma por otra tecnología, el Handler puede seguir usando el contrato del repositorio. Ese es el ejemplo más claro de SRP y DIP.

## 9 Patrones de diseño utilizados

| Patrón            | Uso real                                                  | Por qué se utilizó                                               |
|-------------------|-----------------------------------------------------------|------------------------------------------------------------------|
| CQRS              | Commands para crear o actualizar y Queries para consultar | Separar lectura y escritura y mantener casos de uso pequeños.    |
| Repository        | NoteRepository y BinnacleRepository                       | Aislar reglas de negocio de Prisma y MySQL.                      |
| Facade            | NoteFacade y BinnacleFacade                               | Dar a la pantalla una API simple y coordinar API y Store.        |
| Mapper            | Prisma Mapper y Response Mapper                           | Evitar que formatos de base de datos se filtren a la UI.         |
| DTO               | Create y Update DTO                                       | Definir y validar la entrada de cada endpoint.                   |
| Reactive Store    | Signals y Stores                                          | Mantener estado predecible de carga, datos y errores.            |
| Guard y Decorator | JWT CurrentUser y Roles                                   | Centralizar autenticación y autorización.                        |
| Soft delete       | status igual a cero                                       | Conservar trazabilidad y auditoría sin borrado físico inmediato. |

## Por qué no son patrones decorativos

Cada patrón resuelve un problema presente en el mantenimiento. CQRS organiza las operaciones; Repository desacopla persistencia; Facade reduce dependencias de la pantalla; Mapper protege los límites entre capas; DTO evita datos inválidos; los Guards impiden acceso no autorizado. No se añadieron solo para nombrarlos en la exposición.

## 10 Puntualidad

La entrega indicada vence el 23 de septiembre de 2026 a las 23:59. La puntualidad no se demuestra únicamente diciendo que se trabajó a tiempo: debe respaldarse con el archivo entregado, el registro de envío y una demostración preparada antes del cierre.

| Evidencia | Estado | Acción |
|-----------|--------|--------|
|-----------|--------|--------|

| Evidencia             | Estado                         | Acción                                                 |
|-----------------------|--------------------------------|--------------------------------------------------------|
| Funciones principales | Implementadas                  | Demostrar Notas y Bitácora de principio a fin.         |
| Compilación frontend  | Correcta                       | Conservar la salida del build como evidencia.          |
| Compilación backend   | Correcta                       | Conservar la salida de Prisma y Nest build.            |
| Documento de defensa  | Preparado                      | Adjuntar este informe antes de las 23:59.              |
| Envío en plataforma   | Responsabilidad del estudiante | Confirmar el envío y guardar captura con fecha y hora. |

## 11 Guion para presentar y defender

| Tiempo | Tema             | Qué mostrar o explicar                                                |
|--------|------------------|-----------------------------------------------------------------------|
| 1 min  | Alcance          | Notas con URL, tareas, imágenes y editor; Bitácora con URL y tareas.  |
| 2 min  | Notas            | Agregar URL y tarea, editar imagen, guardar, recargar y marcar tarea. |
| 2 min  | Bitácora         | Crear, abrir Ver, marcar tarea, entrar a Editar y guardar.            |
| 2 min  | Código           | Modelos, FormData o JSON, DTO, Handler, Repository y schema Prisma.   |
| 2 min  | Calidad          | Validaciones, permisos, límites, build correcto y mejoras pendientes. |
| 2 min  | SOLID y patrones | SRP, DIP, CQRS, Repository, Facade, Mapper y DTO.                     |
| 1 min  | Cierre           | Valor entregado y estado de la solución.                              |

### 11.1 Demostración paso a paso

1. Iniciar sesión y abrir una nota existente.
2. Entrar a Editar, agregar una URL sin protocolo y comprobar que se normaliza.
3. Agregar una tarea y seleccionar una imagen JPEG, PNG o WebP.
4. Abrir el editor, dibujar o agregar texto, deshacer y guardar la imagen editada.
5. Guardar la nota, recargar y comprobar enlaces, archivos y tareas.
6. Marcar una tarea desde Ver sin entrar de nuevo a Editar.
7. Abrir Bitácora, crear un registro con URL y tareas.
8. Entrar al detalle, marcar una tarea desde Ver y recargar.
9. Mostrar que Ver y Editar son estados separados.
10. Cerrar enseñando schema Prisma, Handler y Repository.

## 11.2 Mensaje de apertura

El mantenimiento asignado amplió dos módulos existentes. En Notas incorporamos enlaces URL, tareas, imágenes y un editor gráfico. En Bitácora agregamos enlaces y tareas, además de separar la consulta de la edición. La solución no se limitó a la interfaz: incluye validación, permisos, casos de uso y persistencia en MySQL.

## 11.3 Mensaje de cierre

El resultado permite relacionar información externa, seguir actividades y trabajar visualmente con imágenes sin salir de la nota. En Bitácora, el usuario consulta primero y edita solo cuando lo necesita, pero puede completar tareas de forma directa. La arquitectura mantiene las responsabilidades separadas, protege los datos y permite continuar mejorando cada módulo.

# 12 Preguntas probables y respuestas

### ¿Dónde se guardan las tareas y los enlaces?

En los campos JSON opcionales tasks y links de Note y Binnacle en MySQL, accedidos mediante Prisma.

### ¿Por qué JSON y no tablas nuevas?

Son listas pequeñas, dependientes de un solo registro y se recuperan siempre junto con él. JSON redujo complejidad para este alcance; si luego requieren consultas globales o relaciones propias convendría normalizarlas.

### ¿Cómo validan una URL?

Angular normaliza y usa new URL; el backend vuelve a validar. En Bitácora class validator exige HTTP o HTTPS con protocolo.

### ¿Cómo evitan editar datos ajenos?

El JWT entrega el userId. Bitácora compara ese id con el propietario; Notas valida propietario o membresía del repositorio según su visibilidad.

### ¿Qué pasa al marcar una tarea desde Ver?

La señal actualiza la lista y persist false ejecuta PUT sin activar el editor. El servidor devuelve el registro actualizado.

### ¿Por qué Fabric.js?

Resuelve selección, objetos, dibujo, serialización e interacción sobre Canvas, evitando implementar un motor gráfico manual.

### ¿Dónde está SOLID?

En la separación Component, Facade, API, Handler, Repository y Mapper; además los Handlers dependen de contratos de repositorio.

### ¿Qué patrones usaron?

CQRS, Repository, Facade, Mapper, DTO, Store reactivo, Guards y soft delete.

### ¿Cómo demostraron calidad?

Con validaciones en dos capas, permisos, límites, manejo de errores y compilaciones correctas de Angular y NestJS.

### ¿Qué falta mejorar?

Pruebas automatizadas específicas, reducción del SCSS que supera el presupuesto y pruebas adicionales de archivos grandes y fallos de red.

## 13 Conclusión

El mantenimiento cubre el alcance funcional solicitado en Notas y Bitácora y lo integra con la arquitectura existente. Las mejoras tienen contratos tipados, validación, autorización, persistencia y respuesta visual. Las compilaciones pasan; el siguiente paso de calidad es incorporar pruebas automatizadas y atender la advertencia de tamaño de estilos.

## Anexo de archivos de referencia

- prx-frontend/src/app/features/notes
- prx-frontend/src/app/features/binnacles
- prx-backend/src/modules/notes
- prx-backend/src/modules/binnacles
- prx-backend/prisma/schema.prisma